

ACTIVITAT AVALUABLE AC3

PRÁCTICA GUIADA: COPILOT PROFESSIONAL (PARTE 2 – TESTING CON IA)

REQUISITOS PREVIOS

Intalar dependencias para test:

```
npm init -y
npm install -D vitest jsdom @testing-library/dom @testing-library/user-event
playwright
```

Añadir al proyecto (package.json) los scripts de test:

```
"scripts": {
  "test": "vitest",
  "test:ui": "vitest --ui",
  "test:e2e": "playwright test"
}
```



```
{
  "name": "mpropi_u3_ac3",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "vitest",
    "test:ui": "vitest --ui",
    "test:e2e": "playwright test"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/hakkai30/MPROPI_U3_AC3.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/hakkai30/MPROPI_U3_AC3/issues"
  },
  "homepage": "https://github.com/hakkai30/MPROPI_U3_AC3#readme",
  "description": "",
  "devDependencies": {
    "@testing-library/dom": "^10.4.1",
    "@testing-library/user-event": "^14.6.1",
    "jsdom": "^29.1.0",
    "playwright": "^1.59.1",
    "vitest": "^4.1.5"
  }
}
```

A. TESTING UNITARIO

Fragmento teórico

El test unitario verifica una unidad aislada de código (normalmente una función).

Características:

- no depende del DOM ni del backend
- ejecución rápida
- detecta errores lógicos
- fácil de automatizar

Ejemplo típico:

- validar inputs
- calcular valores
- comprobar condiciones

Limitación clave:

- no detecta errores de integración (frontend/backend)

EJERCICIO A1. Test unitario básico

Vamos a testear la función validateLogin en shared/validation.js

Acción

Crear validation.test.js:

```
import { describe, it, expect } from 'vitest'
import { validateLogin } from '../validation'

describe('validateLogin', () => {
  it('fails if email is empty', () => {
    expect(validateLogin('', '12345678').valid).toBe(false)
  })
  it('fails if password is empty', () => {
    expect(validateLogin('test@test.com', '').valid).toBe(false)
  })
  it('passes with valid input', () => {
    expect(validateLogin('test@test.com', '12345678').valid).toBe(true)
  })
})
```

Cómo ejecutar

```
npm run test
```

(Se va a ejecutar el script test del package.json(comando vitest) sobre todos los archivos *.test que se encuentren en el proyecto.

Qué observar

- tests rápidos
- resultado inmediato
- lógica validada

```

robin@Robins-MacBook-Air MPROPI_U3_AC3 % npm run test
> mpropi_u3_ac3@1.0.0 test
> vitest

DEV v4.1.5 /Users/robin/Desktop/DAW2/BIG DATA/MPROPI_U3_AC3

✓ MPROPI-U3-AC3/shared/validation.test.js (3 tests) 2ms
  ✓ validateLogin (3)
    ✓ fails if email is empty 1ms
    ✓ fails if password is empty 0ms
    ✓ passes with valid input 0ms

Test Files  1 passed (1)
Tests       3 passed (3)
Start at    20:37:45
Duration    86ms (transform 10ms, setup 0ms, import 17ms, tests 2ms, environment 0ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

EJERCICIO A2. Edge cases con IA

Teoría breve

Un edge case es un caso límite:

- valores vacíos
- null / undefined
- formatos incorrectos

Son donde suelen aparecer bugs.

Acción

Fija el scope del agente en validate.test.js y dale el prompt:

```
añade tests unitarios con edge cases
```

Observa cómo se modifica el archivo de test con nuevos casos de test.

Ejecutar

```
npm run test
```

```

import { describe, it, expect } from 'vitest'
import { validateLogin } from './validation'

describe('validateLogin', () => {
  it('fails if email is empty', () => {
    expect(validateLogin('', '12345678').valid).toBe(false)
  })
  it('fails if password is empty', () => {
    expect(validateLogin('test@test.com', '').valid).toBe(false)
  })
  it('passes with valid input', () => {
    expect(validateLogin('test@test.com', '12345678').valid).toBe(true)
  })

  // Edge cases
  it('fails if no arguments are provided', () => {
    const result = validateLogin()
    expect(result.valid).toBe(false)
    expect(result.error).toBe('Campos obligatorios')
  })

  it('fails if arguments are null', () => {
    expect(validateLogin(null, null).valid).toBe(false)
  })

  it('fails if email is null', () => {
    expect(validateLogin(null, '12345678').valid).toBe(false)
  })

  it('fails if password is null', () => {
    expect(validateLogin('test@test.com', null).valid).toBe(false)
  })

  it('fails if email is undefined', () => {
    expect(validateLogin(undefined, '12345678').valid).toBe(false)
  })

  it('fails if password is undefined', () => {
    expect(validateLogin('test@test.com', undefined).valid).toBe(false)
  })

  it('returns the correct error message on empty strings', () => {
    expect(validateLogin('', '').error).toBe('Campos obligatorios')
  })
})

```

```
robin@Robins-MacBook-Air MPROPI_U3_AC3 % npm run test
```

```
> mpropi_u3_ac3@1.0.0 test
> vitest
```

```
DEV v4.1.5 /Users/robin/Desktop/DAW2/BIG DATA/MPROPI_U3_AC3
```

```
✓ MPROPI-U3-AC3/shared/validation.test.js (10 tests) 2ms
  ✓ validateLogin (10)
    ✓ fails if email is empty 1ms
    ✓ fails if password is empty 0ms
    ✓ passes with valid input 0ms
    ✓ fails if no arguments are provided 0ms
    ✓ fails if arguments are null 0ms
    ✓ fails if email is null 0ms
    ✓ fails if password is null 0ms
    ✓ fails if email is undefined 0ms
    ✓ fails if password is undefined 0ms
    ✓ returns the correct error message on empty strings 0ms
```

```
Test Files 1 passed (1)
Tests      10 passed (10)
Start at   22:20:42
Duration   97ms (transform 15ms, setup 0ms, import 27ms, tests 2ms, environment 0ms)
```

```
PASS Waiting for file changes...
press h to show help, press q to quit
```

EJERCICIO A3. Bug real

Teoría breve

El valor real del testing:

→ detectar errores automáticamente al ejecutar

Acción

Vamos a modificar la función `validateLogin` para romper el código y comprobar que ya no se pasan los tests unitarios.

```
if (!email && !password)
```

Ejecutar

```
npm run test
```

Resultado

- tests fallan
- bug detectado

Acción

Recuerda volver a dejar la función `validateLogin` en su forma correcta.

```
export function validateLogin(email, password) {  
  if (!email && !password) {  
    return { valid: false, error: 'Campos obligatorios' }  
  }  
  
  return { valid: true }  
}
```

```
FAIL MPROPI-U3-AC3/shared/validation.test.js > validateLogin > fails if password is undefined  
AssertionError: expected true to be false // Object.is equality  
  
- Expected  
+ Received  
  
- false  
+ true  
  
> MPROPI-U3-AC3/shared/validation.test.js:39:61  
37|  
38|   it('fails if password is undefined', () => {  
39|     expect(validateLogin('test@test.com', undefined).valid).toBe(false)  
   |                                                                ^  
40|   })  
41|  
  
Test Files   1 failed (1)  
Tests       6 failed | 4 passed (10)  
Start at    22:26:50  
Duration    14ms  
  
FAIL Tests failed. Watching for file changes...  
press h to show help, press q to quit
```

B. TEST FUNCIONAL (SIMULACIÓN DE DEPENDENCIAS)

Fragmento teórico

El test funcional verifica cómo se comporta el sistema desde el punto de vista del usuario.

Características:

- interactúa con el DOM
- simula acciones (click, escribir)
- valida comportamiento visible

Ejemplo:

- escribir en input
- hacer click
- ver mensaje

Diferencia con unitario:

- unitario → lógica interna
- funcional → comportamiento externo

EJERCICIO B1. Test funcional básico (frontend)

Acción

Crea el archivo frontend/app.test.js con el siguiente contenido

```
import userEvent from '@testing-library/user-event'
import { test, expect } from 'vitest'

test('user interaction', async () => {
  // 1. Crear DOM
  document.body.innerHTML = `
    <input id="email" />
    <input id="password" />
    <button id="loginBtn">Login</button>
    <p id="message"></p>
  `
  const email = document.getElementById('email')
  const password = document.getElementById('password')
  const button = document.getElementById('loginBtn')
  await userEvent.type(email, 'test@test.com')
  await userEvent.type(password, '12345678')
  await userEvent.click(button)
})
```

Cómo ejecutar

```
npm run test
```

Recuerda que Vitest detecta automáticamente:

- archivos *.test.js
- dentro de cualquier carpeta

Qué observar

⚠ Problema muy importante

Este test no funcionará completamente por varios motivos:

- app.js usa fetch: hay que simular ese proceso para poder hacer el test, pero al estar simulando sólo el frontend, no hay backend real. Debemos hacer un mock del fetch.

No hay en app.js ninguna función testeable, para ello hay que modificar el archivo así para exportar la función:

```
export function initApp() {
  document.getElementById('loginBtn').addEventListener('click', async () => {
    const email = document.getElementById('email').value
    const password = document.getElementById('password').value
    const response = await fetch('http://localhost:3000/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    })
    const data = await response.json()
    document.getElementById('message').textContent = data.message
  })
}
```

Al estar ejecutando el test en Node y no en el navegador, NO HAY DOM. Debemos pedirle a Vitest que lo simule.

Para ello, añadimos el archivo vitest.config.js a la raíz del proyecto, con este contenido:

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    environment: 'jsdom'
  }
})
```

Aplicados estos cambios, el test app.test.js queda así:

```
// 0. Importamos JSDOM para simular DOM
// @vitest-environment jsdom
import { test, expect, vi } from 'vitest'
import userEvent from '@testing-library/user-event'
import { initApp } from './app.js'

test('login correcto', async () => {
  // 1. Crear DOM
  document.body.innerHTML = `
    <input id="email" />
    <input id="password" />
    <button id="loginBtn">Login</button>
    <p id="message"></p>
  `
  // 2. Inicializar app
  initApp()
  // 3. Mock fetch
  global.fetch = vi.fn(() =>
    Promise.resolve({
      json: () => Promise.resolve({ message: 'Login correcto' })
    })
  )
  // 4. Simular usuario
  await userEvent.type(document.getElementById('email'), 'test@test.com')
  await userEvent.type(document.getElementById('password'), '12345678')
  await userEvent.click(document.getElementById('loginBtn'))
  // 5. Verificar
  expect(document.getElementById('message').textContent).toBe('Login correcto')
})
```

👉 Ahora sí que el test debe pasarse exitosamente.

Esto nos sirve para ver que:

- escribir test ≠ test válido
- falta simular dependencias

```
✓ MPROPI-U3-AC3/frontend/app.test.js (1 test) 97ms
✓ login correcto 97ms

Test Files  1 passed (1)
Tests      1 passed (1)
Start at   22:34:17
Duration   179ms

PASS Waiting for file changes...
press h to show help, press q to quit
```

EJERCICIO B2. Error de usuario

Teoría breve

Los tests funcionales deben cubrir:

- casos correctos
- casos de error

Acción

Con el contexto fijado en app.test.js, escribe el prompt:

```
genera test donde login falla
```

Ejecutar

```
npm run test
```

Observa que pasa el test.

```
RERUN MPROPI-U3-AC3/frontend/app.test.js x2

✓ MPROPI-U3-AC3/frontend/app.test.js (2 tests) 166ms
  ✓ login correcto 117ms
  ✓ login falla 48ms

Test Files  1 passed (1)
Tests       2 passed (2)
Start at    22:36:09
Duration    248ms

PASS Waiting for file changes...
        press h to show help, press q to quit
```

C. TEST END-TO-END (E2E)

Fragmento teórico

El test E2E (end-to-end) prueba el sistema completo:

- frontend
- backend
- red

Simula un usuario real en un navegador.

Características:

- más lento
- más realista
- detecta errores de integración

Ejemplo:

- login completo
- navegación
- flujo de usuario

EJERCICIO C1. Error de usuario

Acción:

Prompt al agente:

```
Start the backend server, open the frontend in a browser, and test the login flow
with three different credential pairs:
1. correct credentials (test@test.com / 12345678)
2. wrong password
3. empty fields
Verify the UI message for each case and report the results.
Show the interaction with an integrated browser.
```

Observa cómo:

- El agente te pide permiso para instalar dependencias faltantes del servidor (express).
- El agente te pide permiso para instalar (si es la primera vez) las tools necesarias para el test E2E (chromium, etc).
- El agente levanta la app en el localhost.
- El agente abre un navegador integrado en el IDE (chromium). LOL.
- El agente simula la interacción con la app web (DOM) y lee la respuesta del servidor.
- Se presentan los resultados del test.

Login

Login

```
# Login Flow Testing Plan

- [x] Navigate to http://localhost:3000
- [x] Test Case 1: Correct credentials (test@test.com / 12345678)
- [x] Test Case 2: Wrong password (test@test.com / wrongpassword)
- [x] Test Case 3: Empty fields
- [x] Final Report Summary

## Findings
- **Correct Credentials:** Login correcto
- **Wrong Password:** Credenciales incorrectas
- **Empty Fields:** Credenciales incorrectas
```

D. REVIEWS (AUDITORÍAS)

EJERCICIO D1. Review de seguridad

Prompt al chat:

```
@workspace Review security issues
```

? Brechas esperadas:

- passwords harcodeadas en el código ✖
- CORS no configurado ✖
- No hay HTTPS ✖
- No hay rate limiting en los endpoints del servidor ✖
- (...)

Auditoría de Seguridad del Workspace

Tras revisar el código de tu aplicación (Frontend, Backend y código compartido), he identificado varios problemas de seguridad que deberías abordar antes de considerar esta aplicación lista para un entorno de producción.

🚨 Vulnerabilidades Críticas

1. Credenciales Hardcodeadas y Contraseñas en Texto Plano

- ****Ubicación:**** `backend/server.js` (Línea 16)
- ****Problema:**** La validación se hace comparando directamente con valores fijos:
`email === 'test@test.com' && password === '12345678'`.
 - Nunca se deben incluir credenciales en el código fuente.
 - Las contraseñas nunca deben guardarse ni compararse en texto plano.
- ****Remediación:**** Utilizar una base de datos. Las contraseñas deben ser hasheadas usando algoritmos criptográficos robustos (como `bcrypt` o `argon2`) y un "salt" único por usuario. La comparación debe hacerse comparando los hashes.

🔴 Vulnerabilidades Altas

2. Transmisión de Datos Insegura (HTTP)

- ****Ubicación:**** `frontend/app.js` (Línea 5) y configuración del servidor.
- ****Problema:**** Las peticiones se hacen a `http://localhost:3000`. Al usar HTTP en lugar de HTTPS, las credenciales viajan en texto plano por la red y pueden ser interceptadas mediante ataques de "Man-in-the-Middle" (MitM).
- ****Remediación:**** Configurar un certificado SSL/TLS en el servidor para producción de manera que todo el tráfico viaje encriptado sobre HTTPS.

3. Falta de Protección contra Fuerza Bruta (Rate Limiting)

- ****Ubicación:**** `backend/server.js`
- ****Problema:**** El endpoint `/login` no tiene límite de peticiones. Un atacante podría usar un script para enviar miles de combinaciones de contraseñas por segundo hasta adivinar una válida.
- ****Remediación:**** Implementar un middleware de limitación de tasa (rate-limiting) como `express-rate-limit` para bloquear IPs que hagan demasiados intentos fallidos en un corto periodo de tiempo.

🟡 Vulnerabilidades Medias

4. Validación de Entrada Deficiente e Inconsistente

- ****Ubicación:**** `backend/server.js`, `shared/validation.js`, `frontend/app.js`
- ****Problema:****
 - El backend toma `{ email, password }` directamente de `req.body` sin verificar que sean cadenas de texto (strings) ni limitar su longitud (lo que podría llevar a ataques de denegación de servicio por payloads gigantes).
 - La función `validateLogin` en `shared` es muy débil: solo comprueba que los valores existan (`!email || !password`), pero no verifica que el email tenga un formato válido ni que la contraseña tenga una longitud mínima/máxima.
 - El frontend ****no llama**** a `validateLogin` antes de enviar la petición.
- ****Remediación:****
 - Integrar la función `validateLogin` en `app.js` antes de hacer el `fetch`.
 - Mejorar la validación utilizando expresiones regulares para el email y límites de longitud para la contraseña.
 - Re-validar los datos estrictamente en el backend (ej. usando librerías como `zod` o `joi`).

5. Inexistencia de Gestión de Sesiones (Autenticación)

- ****Ubicación:**** `backend/server.js`
- ****Problema:**** Tras un login exitoso, el servidor simplemente devuelve un mensaje `{"message": "Login correcto"}`. No se emite ninguna prueba criptográfica de identidad (como un JWT) ni se establece una cookie de sesión.
- ****Remediación:**** Tras validar las credenciales, el servidor debe generar un JSON Web Token (JWT) o una Cookie de sesión `HttpOnly` y `Secure` para que el frontend pueda autenticarse en peticiones futuras.

🟢 Mejoras de Buenas Prácticas (Bajas)

6. Ausencia de Cabeceras de Seguridad (Security Headers)

- ****Ubicación:**** `backend/server.js`
- ****Problema:**** Express por defecto envía cabeceras como `X-Powered-By: Express` que revelan tecnología del backend y omite cabeceras de seguridad modernas (HSTS, X-Content-Type-Options, etc).
- ****Remediación:**** Instalar y utilizar el middleware `helmet` (`app.use(helmet())`).

EJERCICIO D2. Review de accesibilidad

Opción 1 (bàsica)

```
@file index.html Review accessibility (WCAG)
```

Opción 2 (refinada con agent skills)

Objectiu

Dar contexto externo estructurado (skills) a Copilot

Idea clau

Copilot sense context:

- superficial

Copilot amb context estructurat:

- molt millor

PAS A PAS — CREAR FITXER DE “SKILLS WCAG”

1. Crear fitxer

 wcag-guidelines.md

2. Generar skills con Copilot (chat)

PROMPT: Generate a concise developer-friendly WCAG checklist for HTML forms

3. Usar aquest fitxer com a context

 Selecciona el contingut del fitxer (ratolí) i copiar-lo en el fichero wcag-guidelines.md

4. Prompt amb context

PROMPT: Apply these WCAG guidelines to index.html

5. Alternativa amb referència

PROMPT: @file wcag-guidelines.md @file index.html Improve accessibility

Compara els resultats del review sense i amb skills.

RESULTAT

Edited index.html

He actualizado `frontend/index.html` siguiendo las pautas establecidas en tu guía WCAG.

Aquí tienes un resumen de las mejoras de accesibilidad que he implementado en el archivo:

1. ****Estructura y Semántica Base:****

- Añadí la etiqueta ``<html lang="es">`` para que los lectores de pantalla sepan en qué idioma está el contenido.
- Incluí un ``<head>`` con un ``<title>`` descriptivo ("Inicio de Sesión"), fundamental para navegación no visual.
- Envolví el contenido principal en una etiqueta ``<main>``.

2. ****Etiquetas (Labels) Explícitas:****

- Agregué ``<label for="email">`` y ``<label for="password">`` vinculadas a los `id` de sus respectivos inputs, asegurando que un clic en el texto seleccione el input y los lectores de pantalla anuncien correctamente en qué campo está el usuario.

3. ****Tipos de Input y Autocompletado:****

- Cambié el campo de correo a `type="email"` para que se abra el teclado numérico correcto en móviles.

- Añadí los atributos `autocomplete="email"` y `autocomplete="current-password"` para facilitar a los usuarios el rellenado de los campos mediante el navegador (útil tanto para agilizar como para facilitar su uso a personas con problemas cognitivos).

4. ****Validación (ARIA):****

- Marqué explícitamente los campos obligatorios usando `required` nativo y `aria-required="true"`.

- Incluí un pequeño asterisco visual (`\*`) envuelto con `aria-hidden="true"` para que no ensucie la narración en el lector de pantalla.

5. ****Mensajes Dinámicos (Live Regions):****

- Agregué `role="status"` y `aria-live="polite"` al párrafo de mensajes (`<p id="message">`). Con esto, cuando la respuesta del servidor se pinte dinámicamente ("Login correcto" o "Credenciales incorrectas"), el lector de pantalla leerá el mensaje automáticamente al usuario.